

Relatório do Trabalho Final - Etapa 2

Prof. Weverton Cordeiro

Eduardo Fonseca da Silva, Estevan Zanetti Küster

1. Descrição do Ambiente de Testes

Os testes e validações foram executados em ambientes diferentes.

- **Sistemas Operacionais:** Windows 11 (Nativo), Linux via WSL 2 (Ubuntu 22.04 LTS) e instâncias Linux Nativas nos laboratórios da universidade.
- **Linguagem e Compiladores:** Embora a especificação da etapa 2 sugerisse C/C++, optamos por desenvolver a solução em Python 3, conforme estabelecido previamente em sala de aula. Não foram necessários compiladores externos. Utilizamos o interpretador padrão CPython e apenas bibliotecas da *Standard Library* (*socket* para rede UDP, *threading* para concorrência e *json/hashlib* para serialização).

2. Arquitetura do Cluster e Componentes Principais

A aplicação funciona com um estado replicado em memória e comunicação UDP. Cada cliente cria um UUID novo ao iniciar e envia uma requisição por vez. Em caso de perda, o cliente retransmite a requisição. O novo líder responde com o ACK previamente replicado ou processa a operação uma única vez.

O **estado replicado** do sistema inclui:

- A quantidade global de requisições e a soma total;
- A versão global do estado (*state_version*);
- O termo de eleição e versão de *membership*;
- O último *request* e último ACK de cada UUID cliente;
- O último endereço conhecido de cada cliente.

A. Máquina de Estados (Papéis Possíveis de um Nó)

Um nó do cluster é uma entidade que altera seu estado em resposta a eventos da rede e *timeouts*. Os estados possíveis são:

- **JOINING:** O nó recém-ligado. Não faz nada além de transmitir mensagens de *SERVER_HELLO* até que um Líder o note e envie um *Snapshot* com os dados atualizados para integrá-lo ao cluster.
- **BACKUP:** O nó passivo e sincronizado. Processa comandos de replicação enviados pelo Líder e emite *BACKUP_HEARTBEAT* para avisar que está vivo. Se deixar de receber o

heartbeat do Líder, presume falha e transita para CANDIDATE.

- **CANDIDATE:** O nó que detectou ausência de liderança. Incrementa seu Relógio Lógico (Termo de Eleição) e envia mensagens ELECTION para nós com ID maior. Se um nó maior responder, ele recua. Se não houver oposição, ele se declara Líder.
- **PRIMARY:** O líder da rede. É o único autorizado a processar CLIENT_REQUEST. Ele serializa as operações, força a replicação nos backups e coordena a expulsão de nós inativos.
- **LEAVING:** O nó que recebeu um comando de desligamento gracioso (stop()). Ele avisa a rede, transfere a liderança se necessário, e encerra o processo.

B. Threads de Processamento

O nó opera com base em duas rotinas assíncronas atuando em paralelo:

- **receive_loop (Recepção):** *Thread* bloqueante que escuta o socket UDP. Ao receber um pacote, decodifica o JSON e o entrega ao roteador central, que aciona a função de tratamento apropriada.
- **maintenance_loop (Metabolismo):** *Loop* rodando a cada 0.1s. Gerencia os timeouts globais: dispara HEARTBEAT ou BACKUP_HEARTBEAT, executa verificações para remover nós lentos e emite logs de status para monitoramento.

3. Decisões Arquiteturais e Fluxo de Dados

A. Algoritmo de Eleição de Líder (Valentão modificado)

O sistema utiliza o Algoritmo do Valentão (Bully) aliado a *Relógios Lógicos (Termos)* e *Versões de Estado*.

- **Justificativa da Modificação:** Para evitar que um nó com dados velhos vença a eleição apenas por ter um ID maior, a métrica suprema do cluster tornou-se a *state_version* (quantas operações o banco já processou). Um nó com versão inferior *já* pode se tornar líder.

B. Replicação Passiva

Adotamos o modelo Primary-Backup. A justificativa para o uso da replicação passiva é que esta é uma forma simples e direta de garantir a consistência do estado entre todos os nós da rede.

O fluxo de uma requisição ocorre em 5 etapas:

1. **Recepção:** Cliente envia requisição ao PRIMARY.
2. **Validação:** O PRIMARY adquire *locks* de commit para garantir processamento sequencial. Checa se é um comando novo, duplicado ou um comando do futuro (buraco no estado).
3. **Replicação Prévia:** O PRIMARY envia a operação (REPLICATION) para todos os BACKUPS ativos e aguarda.
4. **Confirmação:** A função pausa até receber REPLICATION_ACK dos backups.
5. **Retorno:** Após consenso da rede, o PRIMARY atualiza sua versão e responde ao cliente (CLIENT_ACK).

C. O Botão de Pânico

Para evitar impasses, o sistema não tenta resolver anomalias suavemente. Se uma operação crucial demorar (timeout replicando, timeout esperando snapshot, detecção de buracos nos dados), o nó invoca um "Botão de Pânico" (`schedule_recovery`). Ele aborta a transação instantaneamente, loga o erro e dispara uma nova eleição em 100ms. O sistema prefere reiniciar rapidamente do que ficar em um estado incerto ou paralisado.

4. Desafios e Resolução de Conflitos

O principal desafio enfrentado durante os testes foi sincronizar o cluster quando dois líderes se encontravam após uma queda de rede.

Quando um nó perdia a conexão com a rede, ele se declarava líder (aumentando seu Termo de eleição), mas acabava ficando com o banco de dados desatualizado por estar isolado. Ao se reconectar, o Líder Real (que possuía dados atualizados, porém um Termo menor) abdicava do trono obedecendo à regra cega de termos maiores. Isso gerava um impasse (*livelock*), pois o "falso líder" desatualizado tentava sincronizar dados, falhava no processo e acionava infinitas eleições, paralisando totalmente o atendimento do cluster.

Resolução: Múltiplas Camadas

Para resolver esse conflito, implementamos uma combinação de estratégias tanto no servidor quanto no cliente:

1. **Versão como Fonte da Verdade e Roubo de Termo:** O Líder Real parou de recuar apenas por causa do Termo de eleição. Ele passou a avaliar primariamente a versão dos dados (`state_version`). Se o usurpador tem um Termo maior, mas dados obsoletos, o Líder Real aplica o **Roubo de Termo**: copia o termo alto para si, soma +1 e convoca uma nova eleição. Com isso, ele esmaga o falso líder instantaneamente e o força a se rebaixar a backup.
2. **O Botão de Pânico:** Aliado a isso, garantimos que se qualquer fluxo de sincronização travar, o "botão de pânico" é acionado para abortar a tentativa de usurpação e puxar uma eleição que restaure a ordem.
3. **Clientes:** O comportamento do cliente foi alterado para ser mais "cético". Ao entrar na rede ou tentar descobrir o líder, ele não aceita a primeira resposta que recebe; ele aguarda um breve momento para coletar todas as mensagens de "sou o líder" e escolhe aquele que apresentar a maior versão de dados. Além disso, se o cliente tentar enviar requisições e receber 3 mensagens de RETRY consecutivas, ele entende que o alvo não é mais confiável e aciona sua rotina de redescoberta do zero.